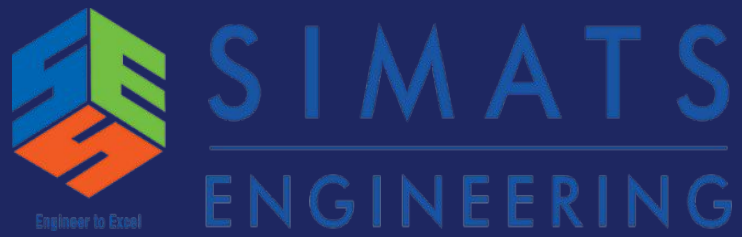


STUDENT MANAGEMENT SYSTEM

Efficient Storage of Marks for 1,000 Students Using
Primitive and Non-Primitive Data Structures

CSA0302- DATA STRUCTURES

GUIDED BY:
PROF.DR.J.ALPHONSA

Presented by:
NAME: [M.Hemakumar]
REGNO:192521138

Introduction

The Challenge

A school or college must record and manage marks for 1,000 students across multiple subjects — accurately, quickly, and at scale.

Why Data Structure Choice Matters

The right structure determines how fast records are accessed, updated, searched, and how efficiently memory is used.

Objective

This presentation identifies an efficient combination of primitive and non-primitive data structures to store and manage student marks.

1,000

Student Records

Fields / student

ID, Name, Marks

Access needed

Fast & repeated

Storage goal

Compact & scalable

Primitive and Non-Primitive Data Structures

Two layers that work together to model a student record

Primitive Data Types

Store a single raw value each

`int`	Roll number / Student ID
`char[]`	Student name
`float`	Marks obtained
`bool`	Pass / fail flag

Role: hold individual, atomic pieces of student information — the raw building blocks of every record.

Non-Primitive Data Structures

Organize many primitive values together

- **Array** — Fixed-size, indexed list of records
- **Structure** — Groups related fields into one unit
- **Linked List** — Dynamic, chained sequence of nodes
- **Tree** — Hierarchical, sorted organization
- **Hash Table** — Key-based fast lookup

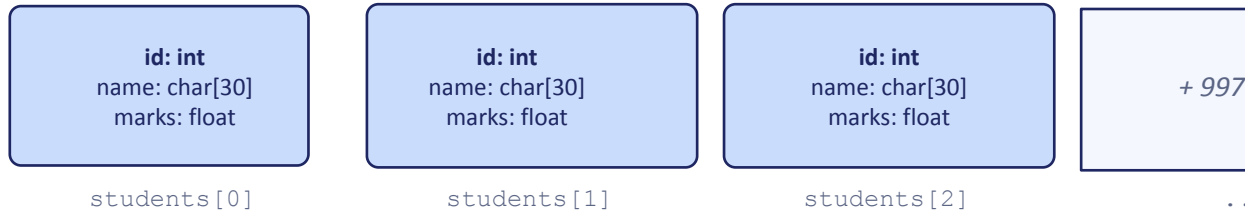
Selected Data Structure: Array of Structures

```
struct Student
{
    int id;
    char name[30];
    float marks;
};

Student students[1000];
```

An Array of Structures groups related fields (primitive types) into one Student record, then repeats that record 1,000 times in a contiguous array.

How it works



Contiguous block in memory → index gives direct (O(1)) access to any record

Primitive + Non-Primitive, Working Together

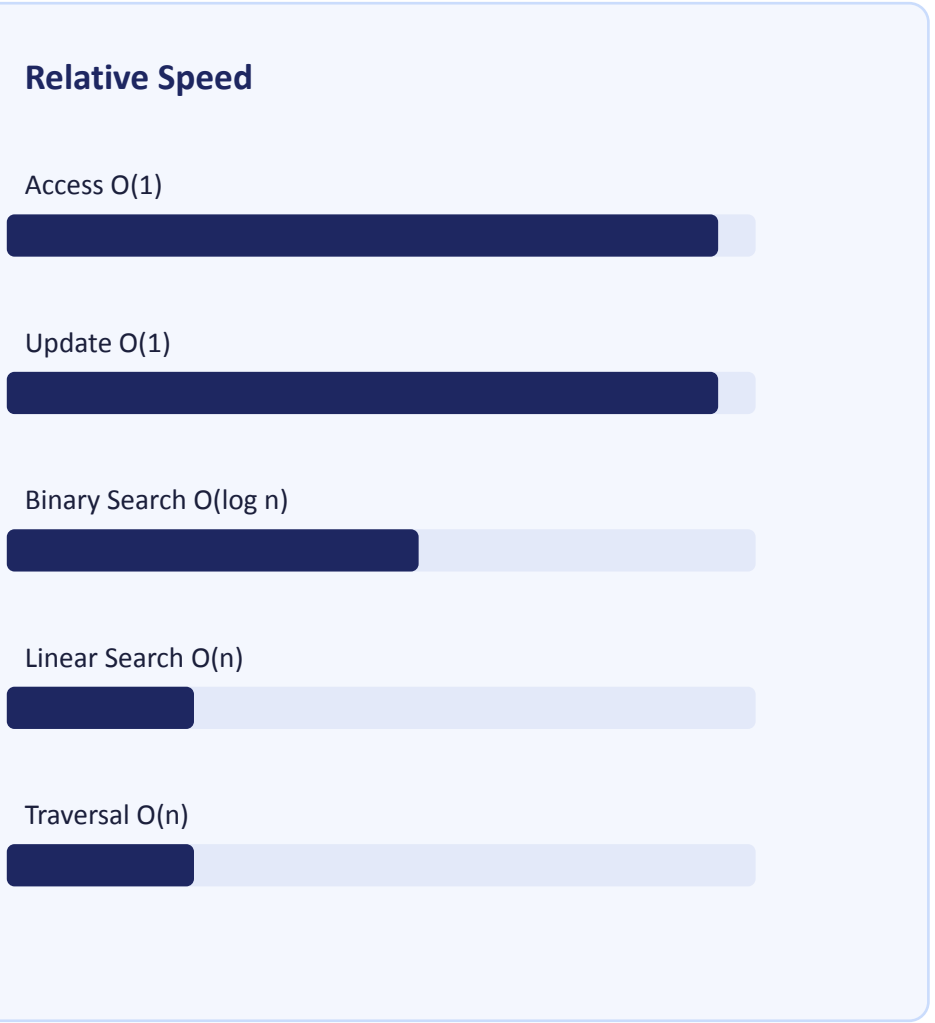
- **Structure:** bundles int, char[], float into one Student record
- **Array:** stores 1,000 such records contiguously
- **Result:** each mark is reachable by a single index lookup

Performance Efficiency

Operation	Time Complexity
Access	`O(1)`
Update	`O(1)`
Traversal	`O(n)`
Linear Search	`O(n)`
Binary Search (Sorted Array)	`O(log n)`

Why arrays excel for a fixed 1,000-student dataset

Because the student count is fixed and known in advance, an array avoids the overhead of dynamic structures like linked lists. Contiguous memory means the address of any record is computed instantly from its index — no traversal required for access or update.



Justification: Why Array of Structures?

Structure	Access	Memory	Fits Fixed Marks Data?
Array (of Structures)	**O(1)**	**Compact, contiguous**	**Best fit**
Linked List	O(n)	Extra pointer overhead	Slower access
Stack	O(1) top only	Contiguous / linked	Not designed for lookup
Queue	O(1) ends only	Contiguous / linked	Not designed for lookup
Tree	O(log n)	Extra pointers/nodes	Overkill for flat records
Hash Table	O(1) avg	Higher memory, hashing cost	Good, but more complex

Array of Structures is the best choice for 1,000 student marks

It offers constant-time access and update, minimal memory overhead with no extra pointers, simple sequential implementation, and predictable performance — exactly matching a fixed-size, record-oriented dataset like student marks.

Conclusion

Advantages Summarized

Fast constant-time access, compact contiguous memory, and a simple, reliable implementation for 1,000 fixed student records.

Two Layers, One System

Primitive data types (int, float, char, bool) store individual values; the array (non-primitive) organizes all 1,000 records into one efficient, indexable collection.

Final Takeaway

The Array of Structures delivers fast access, efficient memory usage, and easy implementation — the optimal structure for this problem.